UNIVERSIDADE DE SÃO PAULO - ICMC
DEPARTAMENTO DE CIÊNCIAS DE COMPUTAÇÃO
Bacharelado em Sistemas de Informação
SCC0282 Recuperação de Informação - 2º Sem/2026
Prof. Marcelo Manzato (mmanzato@icmc.usp.br).

Trabalho Prático 2

Divulgação: 02/10/2026.
Data de entrega: 19/11/2026 até às 23:59.
Grupos: preferencialmente, mesma configuração do T1. Até 2 pessoas.

Objetivo

O objetivo geral do projeto é desenvolver e avaliar um sistema de Recuperação de Informação que combine busca lexical e busca semântica, incorporando mecanismos de tolerância a erros, fusão de rankings, reranking e geração de respostas baseada nos documentos recuperados (*Retrieval-Augmented Generation* - RAG). O projeto deverá permitir a investigação experimental das diferenças entre técnicas lexicais e semânticas e da forma como os diferentes componentes de um sistema moderno de Recuperação de Informação interagem.

O pipeline geral do trabalho será: documentos → pré-processamento e indexação → recuperação lexical + recuperação semântica → busca híbrida → reranking → seleção de contexto → LLM → resposta. A simples implementação das técnicas não será suficiente: será necessário realizar avaliação quantitativa, análise por consulta e investigação de casos de sucesso e falha utilizando evidências concretas provenientes dos experimentos.

Ao final deste documento há um roadmap com o pipeline completo e um checklist das etapas. Na figura, a etapa 1 corresponde à preparação do dataset; por isso, a etapa n da figura corresponde ao requisito n−1 deste enunciado (por exemplo, a etapa 9 da figura, Busca híbrida, corresponde ao requisito 8).

Metodologia

Será permitido utilizar APIs e bibliotecas externas para leitura dos textos, tokenização, normalização, remoção de stopwords, stemming, lematização, cálculo de métricas, geração de gráficos, obtenção de embeddings, modelos pré-treinados e execução de LLMs. Entretanto, todos os integrantes deverão compreender integralmente o código entregue, as decisões tomadas e o funcionamento das técnicas utilizadas no projeto.

A construção do índice invertido constitui uma exceção: ela deverá ser implementada obrigatoriamente do zero pelo próprio grupo, conforme detalhado nos requisitos. Poderão ser utilizadas estruturas de dados genéricas da linguagem ou de bibliotecas auxiliares, mas não mecanismos prontos que construam e consultem automaticamente o índice textual.

O conjunto de dados adotado será o **NFCorpus**, na versão disponibilizada pelo benchmark **BEIR**. Deverão ser utilizados os arquivos padronizados da coleção (corpus.jsonl, queries.jsonl e qrels), garantindo que todos os grupos trabalhem sobre a mesma base. O conjunto **dev** poderá ser utilizado para

ajustes de parâmetros; o conjunto **test** deverá ser reservado para a avaliação final. As consultas de cada conjunto são aquelas que possuem julgamentos no respectivo arquivo de qrels (qrels/dev.tsv e qrels/test.tsv): o arquivo queries.jsonl contém as consultas de todos os conjuntos (3.237 no total), e não apenas as 323 do conjunto test. Recomenda-se utilizar o arquivo oficial do BEIR, que já inclui os qrels de train, dev e test; no Hugging Face, os qrels ficam em um repositório separado (BeIR/nfcorpus-qrels). Cada documento possui os campos title e text; salvo justificativa no relatório, ambos deverão ser concatenados para formar a representação do documento em todos os sistemas (lexical, semântico e reranking).

- BEIR - datasets disponíveis: https://github.com/beir-cellar/beir/wiki/Datasets-available
- NFCorpus no Hugging Face: https://huggingface.co/datasets/BeIR/nfcorpus
- NFCorpus - arquivo oficial do BEIR (corpus, queries e qrels): https://public.ukp.informatik.tu-darmstadt.de/thakur/BEIR/datasets/nfcorpus.zip

Os julgamentos de relevância (qrels) deverão ser utilizados para avaliação e, no caso do conjunto dev, para ajuste das configurações explicitamente permitidas. Os qrels do conjunto test não poderão ser usados para modificar consultas, escolher manualmente parâmetros, alterar rankings ou favorecer exemplos específicos. Em particular, a escolha do modelo de embeddings, do modelo de reranking, das etapas de pré-processamento e dos parâmetros do modelo lexical (por exemplo, $k1$ e b do BM25, caso sejam alterados em relação aos valores usuais) deverá ser feita com base no conjunto dev, e não pela comparação de alternativas no conjunto test.

Requisitos

O projeto consiste em desenvolver e analisar os seguintes componentes obrigatórios:

1. Pré-processamento. A leitura dos documentos, tokenização, normalização, remoção de stopwords, stemming, lematização e demais etapas de pré-processamento poderão ser realizadas com bibliotecas externas. O grupo deverá descrever no relatório quais etapas foram utilizadas, as bibliotecas adotadas e as principais decisões tomadas para representar documentos e consultas.

2. Índice invertido. A recuperação lexical deverá obrigatoriamente utilizar um índice invertido implementado do zero pelo próprio grupo. Os alunos poderão escolher as estruturas de dados que considerarem mais adequadas para representar o dicionário de termos e as posting lists. O índice deverá armazenar as informações necessárias ao modelo lexical escolhido, incluindo, quando aplicável, TF, DF, tamanho dos documentos e estatísticas globais. Não será permitido percorrer integralmente todos os documentos da coleção a cada consulta para calcular o ranking. Deverão ser registrados o número de termos do vocabulário, o número total de postings, o tempo de construção do índice e o tempo médio de processamento das consultas.

3. Busca lexical. O grupo deverá escolher uma única alternativa como baseline lexical: Modelo Vetorial, utilizando TF-IDF e similaridade do cosseno, ou BM25. O mesmo modelo lexical deverá ser utilizado nos experimentos subsequentes. Código desenvolvido no Trabalho Prático 1 poderá ser reutilizado e adaptado. A escolha deverá ser informada e brevemente justificada no relatório.

4. Tolerância a erros. O sistema deverá implementar correção ortográfica de termos desconhecidos utilizando k-gramas para geração de candidatos, coeficiente de Jaccard para filtragem e distância de edição para seleção das alternativas mais próximas. Deverão ser utilizadas 10 consultas do conjunto de teste, nas quais o grupo introduzirá erros ortográficos envolvendo diferentes tipos de alteração (inserção, remoção, substituição ou troca da ordem de caracteres). Para cada consulta deverão ser comparadas três condições: consulta original, consulta com erro e consulta corrigida. O grupo deverá medir quanto do

desempenho perdido pela introdução do erro foi recuperado pelo mecanismo de correção. Limiares e decisões de parametrização poderão ser ajustados no conjunto dev.

Para padronizar o experimento:

(i) o dicionário utilizado na correção deverá ser formado pelos termos da coleção antes do stemming/lematização;
(ii) cada erro introduzido deverá gerar um termo inexistente no dicionário (erros que produzem outra palavra válida não são detectados por este método e não deverão ser usados);
(iii) deverão ser escolhidas consultas cujo NDCG@10 na versão original seja maior que zero;
(iv) a avaliação obrigatória será feita sobre o sistema lexical, utilizando NDCG@10, e a recuperação será calculada por consulta como (NDCG@10 corrigida − NDCG@10 com erro) / (NDCG@10 original − NDCG@10 com erro), quando o denominador for maior que zero; caso contrário, deverá ser registrado que o erro não reduziu o desempenho.

Aplicar as mesmas consultas com erro ao sistema semântico é opcional e pode enriquecer a discussão sobre a robustez de embeddings a erros ortográficos. A correção ortográfica constitui um experimento separado: os demais experimentos deverão utilizar as consultas originais, sem correção.

5. Recuperação semântica. O grupo deverá implementar um mecanismo de recuperação baseado em embeddings densos, utilizando um modelo pré-treinado com arquitetura bi-encoder. Os embeddings dos documentos deverão ser calculados previamente e armazenados; para cada consulta será gerado seu embedding e calculada a similaridade com os documentos. Não será necessário implementar, treinar ou realizar fine-tuning de um modelo neural do zero. Bibliotecas e modelos pré-treinados são permitidos.

Como ponto de partida, recomenda-se a biblioteca Sentence Transformers, que oferece suporte a bi-encoders e busca semântica:

- Documentação/Quickstart: https://www.sbert.net/docs/quickstart.html
- Busca semântica: https://www.sbert.net/examples/sentence_transformer/applications/semantic-search/README.html
- Modelos pré-treinados: https://www.sbert.net/docs/sentence_transformer/pretrained_models.html
- Exemplo - all-MiniLM-L6-v2: https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2
- Exemplo - multi-qa-MiniLM-L6-cos-v1: https://huggingface.co/sentence-transformers/multi-qa-MiniLM-L6-cos-v1

A escolha do modelo é livre. O relatório deverá informar, no mínimo: nome e versão do modelo, biblioteca utilizada, dimensionalidade dos embeddings, função de similaridade, normalização aplicada (quando houver), tempo necessário para gerar os embeddings dos documentos e tempo médio de processamento das consultas. Deverão ser informados também o comprimento máximo de entrada do modelo (em tokens) e se houve truncamento dos documentos. Como a coleção é pequena (3.633 documentos), a busca semântica poderá calcular a similaridade com todos os documentos (busca exaustiva); a restrição de não percorrer toda a coleção a cada consulta aplica-se apenas ao índice invertido da busca lexical.

6. Avaliação da recuperação. Os sistemas lexical e semântico deverão ser avaliados sobre as mesmas consultas do conjunto test. Deverão ser reportadas obrigatoriamente Precision@10, Recall@10, MAP@10 e NDCG@10. Para Precision@10, Recall@10 e MAP@10, considere como relevante todo documento com grau de relevância maior que zero. Para NDCG@10 deverão ser preservados os graus de relevância fornecidos pelos qrels. No caso de MAP@10, deverá ser calculada, para cada consulta, a Average Precision truncada nas dez primeiras posições (AP@10): a soma de Precision@k nas posições $k \leq 10$ que contêm documentos relevantes, dividida pelo número total de documentos relevantes da consulta

nos qrels (convenção do trec_eval e do ir_measures). O MAP@10 é a média desses valores. O Recall@10 também deverá usar como denominador o total de relevantes da consulta nos qrels. Para o NDCG@10, utilize ganho linear (ganho igual ao grau de relevância) e calcule o ranking ideal a partir de todos os julgamentos da consulta, e não apenas dos documentos recuperados. Recomenda-se conferir as métricas implementadas com a biblioteca ir_measures. Além das médias globais, todas as métricas deverão ser preservadas por consulta.

7. Comparação entre busca lexical e busca semântica. Além da comparação quantitativa global, deverá ser realizada uma análise por consulta utilizando a diferença de NDCG@10. Deverão ser identificadas as três consultas com maior vantagem do sistema semântico e as três consultas com maior vantagem do sistema lexical. Para cada uma das seis consultas, apresente o texto da consulta, o Top-5 de cada método, a indicação dos documentos relevantes, os scores ou similaridades e uma análise das razões para a diferença. A discussão deverá procurar evidências relacionadas a correspondência lexical, termos técnicos, sinonímia, paráfrase, variação de vocabulário e ambiguidade.

Afirmações como "o documento era muito longo", "o termo era mais frequente" ou "o modelo favoreceu determinada característica" somente deverão ser feitas quando sustentadas por valores efetivamente observados nos próprios dados.

8. Busca híbrida. Os rankings produzidos pelos sistemas lexical e semântico deverão ser combinados utilizando Reciprocal Rank Fusion (RRF). O parâmetro k deverá ser ajustado no conjunto dev, avaliando pelo menos $k \in \{10, 60, 100\}$, escolhendo-se o valor com maior NDCG@10 médio no dev. O score de um documento d será $RRF(d) = \Sigma\ 1/(k + rank_s(d))$, somando sobre os sistemas s, em que $rank_s(d)$ é a posição de d (começando em 1) no ranking do sistema s; documentos ausentes de um dos rankings não recebem contribuição desse sistema. Deverão ser fundidos os 100 primeiros documentos de cada sistema.

O valor selecionado deverá ser mantido na avaliação final sobre o conjunto test. Deverão ser comparados Lexical, Semântico e Híbrido utilizando Precision@10, Recall@10, MAP@10 e NDCG@10. A utilização de RRF evita a comparação direta de scores produzidos por modelos com escalas diferentes.

9. Reranking. O sistema híbrido será utilizado como recuperador inicial. Os 50 primeiros documentos do ranking híbrido deverão ser reordenados utilizando um modelo mais preciso que considere conjuntamente consulta e documento, preferencialmente um cross-encoder pré-treinado ou método equivalente de reranking neural. Os documentos originalmente abaixo da posição 50 deverão permanecer, após o reranking, na mesma ordem relativa do ranking híbrido. Assim, continua existindo um ranking completo para cálculo das métricas.

Como apoio, podem ser utilizados os recursos de CrossEncoder da biblioteca Sentence Transformers:

- Uso de Cross-Encoders: https://www.sbert.net/docs/cross_encoder/usage/usage.html
- Modelos pré-treinados: https://www.sbert.net/docs/cross_encoder/pretrained_models.html
- Exemplo - cross-encoder/ms-marco-MiniLM-L6-v2: https://huggingface.co/cross-encoder/ms-marco-MiniLM-L6-v2

A escolha do modelo de reranking é livre, mas deverá ser informada e justificada. Deverão ser comparados: Lexical, Semântico, Híbrido e Híbrido + reranking utilizando Precision@10, Recall@10, MAP@10 e NDCG@10. O tempo médio das etapas também deverá ser registrado. Informe também o comprimento máximo de entrada do cross-encoder e se houve truncamento dos documentos.

10. Análise do reranking. Deverão ser identificados os dois casos com maior ganho de NDCG@10 após o reranking, um caso em que houve pouca ou nenhuma alteração e um caso em que o reranking piorou o NDCG@10. Para cada caso, deverão ser mostrados os principais documentos que mudaram de posição e discutida a provável razão da alteração.

11. Retrieval-Augmented Generation (RAG). O projeto deverá incluir obrigatoriamente uma etapa de RAG com o seguinte pipeline: consulta → recuperação híbrida → reranking → seleção de contexto → LLM → resposta. Para manter a correspondência com os qrels, cada item do corpus.jsonl será tratado como uma unidade de contexto no experimento obrigatório. Estratégias adicionais de chunking poderão ser exploradas opcionalmente, desde que seja preservada a relação com o doc_id original.

A LLM será de livre escolha do grupo. Poderão ser utilizados modelos abertos executados localmente ou em ambientes como Google Colab (por exemplo, famílias Qwen, Llama, Gemma ou Mistral), bem como modelos comerciais acessados por API. Não é obrigatório utilizar qualquer serviço pago. O relatório deverá informar modelo e versão, forma de execução (local ou serviço externo), temperatura, número máximo de tokens, demais parâmetros relevantes, custo aproximado caso exista cobrança por API e hardware utilizado no caso de execução local. O uso de um modelo maior ou comercial não implicará automaticamente melhor avaliação do trabalho.

12. Experimento sem RAG × com RAG. Deverão ser utilizadas 10 consultas fixas do conjunto test. Para evitar escolha apenas de casos favoráveis, todos os grupos utilizarão as mesmas consultas, sorteadas com seed 42 entre as 42 consultas do test formuladas como pergunta (texto terminado em "?"), conforme o código abaixo.

As consultas resultantes são: PLAIN-68, PLAIN-2450, PLAIN-153, PLAIN-2850, PLAIN-2750, PLAIN-270, PLAIN-2470, PLAIN-2430, PLAIN-238 e PLAIN-3342; confira se o seu código produz a mesma lista.

Para cada consulta serão executadas duas condições com a mesma LLM e os mesmos parâmetros: (A) sem RAG, em que a LLM recebe somente a pergunta e não possui acesso à coleção NFCorpus; e (B) com RAG, em que a LLM recebe a pergunta e os Top-5 documentos obtidos após o reranking, com seus respectivos identificadores. O prompt do RAG deverá instruir o modelo a responder com base no contexto fornecido, indicar as fontes utilizadas e informar quando não houver evidência suficiente.

Recomenda-se temperatura 0 (ou a menor possível) e, quando disponível, seed fixa, para favorecer a reprodutibilidade. As respostas deverão ser geradas no mesmo idioma nas duas condições, e o idioma escolhido deverá ser informado.

```
import random
# test_qrels: consultas com julgamentos em qrels/test.tsv; queries: textos de
queries.jsonl
ids = [q for q in sorted(test_qrels.keys()) if
queries[q].strip().endswith("?")]
queries_rag = random.Random(42).sample(ids, 10)
```

13. Avaliação do RAG. A avaliação será dividida em duas partes. Na recuperação, deverão ser calculados, para as dez consultas utilizadas no experimento, Recall@5, Precision@5 e Hit@5 (1 se ao menos um documento relevante foi enviado à LLM; 0 caso contrário), verificando se documentos relevantes efetivamente chegaram ao contexto da LLM. Como as consultas do NFCorpus possuem, em média, cerca de 38 documentos relevantes, valores baixos de Recall@5 são esperados e não indicam, por si sós, falha da recuperação.

Na geração, as respostas deverão ser avaliadas por uma avaliação humana estruturada com notas 0, 1 ou 2 segundo os critérios abaixo. Cada nota deverá possuir uma breve justificativa. Recomenda-se que a pontuação seja feita sem identificar a condição (A ou B) de cada resposta, por exemplo embaralhando as respostas antes da avaliação, e, quando possível, que dois integrantes avaliem de forma independente e reportem a concordância entre eles.

Critério	0	1	2
Fidelidade	Afirmações importantes não são sustentadas pelo contexto ou contradizem o contexto.	Resposta parcialmente sustentada.	As afirmações principais são sustentadas pelo contexto.
Completude	Não responde aos aspectos principais.	Responde parcialmente.	Cobre adequadamente os aspectos que podem ser respondidos.
Utilidade	Não responde à necessidade da consulta.	Resposta parcialmente útil.	Responde diretamente à necessidade apresentada.
Rastreabilidade	Não é possível identificar as fontes.	Apenas parte das afirmações pode ser associada às fontes.	As afirmações principais podem ser associadas aos doc_ids apresentados.

Na condição sem RAG não há contexto externo fornecido à LLM, o que deverá ser registrado explicitamente. Nessa condição, a Fidelidade deverá ser avaliada comparando a resposta com os mesmos Top-5 documentos usados na condição com RAG para aquela consulta (isto é, verificando se a resposta gerada sem acesso à coleção é consistente com as evidências disponíveis). A Rastreabilidade deverá ser registrada como "N/A" e excluída da média dessa condição; a ausência de fontes e suas consequências para a verificabilidade da resposta deverão ser discutidas qualitativamente. As médias obtidas nas duas condições deverão ser apresentadas e discutidas.

O objetivo não é demonstrar obrigatoriamente que o RAG sempre produz respostas melhores, mas investigar o efeito da recuperação sobre a qualidade, a fidelidade e a disponibilidade de evidências.

14. Análise de falhas ponta a ponta. Deverão ser analisados detalhadamente três casos, escolhidos entre as dez consultas do experimento: uma consulta em que o RAG melhorou claramente a resposta; uma consulta em que houve pouca diferença; e uma consulta em que o RAG não ajudou ou prejudicou a resposta. No último caso, o grupo deverá investigar em qual estágio ocorreu o problema: recuperação lexical, recuperação semântica, fusão, reranking, seleção de contexto ou geração. Não será suficiente afirmar simplesmente que "a LLM errou"; deverá ser verificado se a informação necessária chegou ou não ao contexto.

15. Arquivos padronizados de resultados. Para facilitar a reprodução e correção, os resultados deverão ser entregues também em arquivos independentes das saídas dos notebooks. No mínimo: results_lexical.csv, results_semantic.csv, results_hybrid.csv, results_reranking.csv, metrics_per_query.csv, typo_experiment.csv e rag_experiment.csv. Os arquivos de ranking deverão conter, no mínimo, query_id, doc_id, rank, score e relevance. O arquivo metrics_per_query.csv deverá conter query_id, model, P10, R10, AP10 e NDCG10. A média de AP10 deverá corresponder ao MAP@10 reportado. Os arquivos de ranking deverão conter os 100 primeiros documentos de cada consulta. O arquivo typo_experiment.csv deverá conter, no mínimo, query_id, query_original, query_erro, query_corrigida, tipo_erro, NDCG10_original, NDCG10_erro e NDCG10_corrigida. O arquivo rag_experiment.csv deverá conter, no mínimo, query_id, query, condition (A ou B), context_doc_ids, prompt, answer, as quatro notas (fidelidade, completude, utilidade e rastreabilidade) e as respectivas justificativas. Resultados existentes apenas como saída de células de notebook não serão considerados como arquivos de resultados entregues.

Análise crítica dos resultados

A análise crítica é parte central do trabalho. Não será suficiente apenas apresentar tabelas de médias ou afirmar que um método foi melhor que outro. Os grupos deverão investigar exemplos concretos, relacionar os resultados aos conceitos estudados e utilizar as análises por consulta solicitadas para explicar por que determinados comportamentos ocorreram.

As comparações deverão alterar apenas o componente que está sendo investigado. Valores de scores produzidos por modelos diferentes não deverão ser comparados diretamente sem justificativa ou normalização adequada. Afirmações sobre frequência de termos, tamanho de documentos, posição no ranking, similaridade ou custo deverão ser acompanhadas de evidências provenientes dos próprios experimentos.

Uso de ferramentas de Inteligência Artificial

É importante distinguir o uso de IA como componente do experimento do uso de IA como ferramenta de apoio ao desenvolvimento. Modelos de embeddings, cross-encoders e LLMs utilizados nas etapas de recuperação semântica, reranking e RAG fazem parte do próprio trabalho e deverão ser documentados na metodologia.

Ferramentas de IA generativa também poderão ser utilizadas como apoio para programação, depuração, explicação de conceitos, elaboração de análises ou revisão textual. Caso sejam utilizadas dessa forma, o relatório deverá conter uma breve seção denominada "Uso de ferramentas de IA", indicando as principais atividades em que foram empregadas. Não é necessário incluir o histórico completo das interações. Todos os integrantes continuam responsáveis por compreender integralmente o código, a metodologia e os resultados apresentados.

Entrega

Deverão ser entregues:

1. **Código-fonte completo.** Todos os scripts, notebooks e módulos necessários para reprodução dos experimentos. Arquivos grandes que podem ser regerados pelos scripts (dataset, embeddings, modelos, índice serializado, ambientes virtuais) não deverão ser incluídos; o README deverá indicar como gerá-los. O código deverá ser entregue no Moodle; links para repositórios não substituem a entrega do arquivo.
2. **README.** Deverá conter integrantes do grupo (nome completo e Nº USP), instruções para instalação e execução, versão da linguagem, bibliotecas e versões principais, modelo lexical escolhido, modelo de embeddings, modelo de reranking, LLM utilizada, hardware empregado e ordem recomendada de execução dos scripts/notebooks.
3. **Resultados completos.** Todos os arquivos CSV padronizados especificados anteriormente, incluídos no arquivo entregue (e não apenas gerados ao executar o código). Chaves de API, senhas ou outras credenciais não deverão ser incluídas na entrega.
4. **Relatório de desempenho.** Relatório em PDF de até 20 páginas, em Português ou Inglês. O limite inclui todo o conteúdo do relatório, exceto apenas as referências bibliográficas. Anexos não deverão ser utilizados para contornar o limite de páginas. O relatório deverá conter as seguintes seções:
   a. Título/autores (nome completo e Nº USP)/filiação/email (cabeçalho)
   b. Introdução (contextualização, motivação e objetivo)
   c. Metodologia (pré-processamento, índice invertido, busca lexical, tolerância a erros, recuperação semântica, busca híbrida, reranking e RAG)
   d. Avaliação (dataset, splits, qrels, métricas, modelos, parâmetros, ambiente computacional e protocolo experimental)
   e. Resultados Obtidos e Análise (lexical × semântico, tolerância a erros, híbrido, reranking, tempos, RAG e análise de falhas)
   f. Considerações finais (principais conclusões obtidas a partir dos experimentos)
   g. Uso de ferramentas de IA

Defesa do projeto

Além da entrega, poderá ser realizada uma breve avaliação individual. Qualquer integrante poderá ser solicitado a explicar uma parte do código, mostrar como o índice invertido foi construído, explicar um cálculo do RRF, interpretar um resultado por consulta, justificar a alteração de posições após o reranking, mostrar quais documentos foram enviados à LLM, explicar por que uma resposta foi considerada fiel ou não, alterar um parâmetro e prever seu efeito ou realizar uma pequena modificação no código ou na configuração experimental.

Observações finais

Os seguintes critérios serão considerados durante a correção dos trabalhos:

1. O índice invertido e a busca lexical foram implementados corretamente e respeitam os requisitos da especificação?
2. A tolerância a erros, a recuperação semântica, a busca híbrida e o reranking foram implementados corretamente?
3. A etapa de RAG foi implementada e avaliada de acordo com o protocolo proposto?
4. A avaliação quantitativa foi realizada corretamente e os resultados são reprodutíveis?
5. As análises utilizam exemplos concretos e evidências extraídas dos próprios experimentos?
6. O relatório possui boa apresentação, organização e qualidade técnica?
7. Os integrantes demonstram compreender o código, os modelos, as métricas e as decisões tomadas durante eventual defesa individual?

Como referência para composição da nota, serão considerados aproximadamente: 10% para indexação e recuperação lexical; 10% para tolerância a erros; 10% para recuperação semântica; 10% para busca híbrida; 10% para reranking; 20% para implementação e avaliação do RAG; 20% para avaliação quantitativa, análise crítica e reprodutibilidade; e 10% para compreensão individual/defesa.

Referências e materiais de apoio

- MANNING, C. D.; RAGHAVAN, P.; SCHÜTZE, H. Introduction to Information Retrieval. Cambridge University Press, 2008. Versão online: https://nlp.stanford.edu/IR-book/information-retrieval-book.html
- BEIR - benchmark e datasets disponíveis: https://github.com/beir-cellar/beir
- BEIR - lista de datasets e NFCorpus: https://github.com/beir-cellar/beir/wiki/Datasets-available
- NFCorpus no Hugging Face: https://huggingface.co/datasets/BeIR/nfcorpus
- Sentence Transformers - documentação: https://www.sbert.net/
- Sentence Transformers - busca semântica: https://www.sbert.net/examples/sentence_transformer/applications/semantic-search/README.html
- Sentence Transformers - Cross-Encoders: https://www.sbert.net/docs/cross_encoder/usage/usage.html
- ir_measures - métricas de avaliação em Recuperação de Informação: https://ir-measur.es/en/latest/

Os materiais acima podem ser utilizados como apoio ao desenvolvimento. A implementação entregue deverá respeitar os requisitos desta especificação, principalmente quanto à construção obrigatória do índice invertido pelo próprio grupo e à compreensão do código utilizado.

T2 — Roadmap do Trabalho Prático 2

Recuperação Semântica, Busca Híbrida, Reranking e RAG

1 Base do Projeto

Preparar os dados e implementar a base de busca lexical.

1 Dataset e splits

- Usar NFCorpus (BEIR)
- Arquivos: corpus.jsonl, queries.jsonl, qrels
- Usar dev para ajuste de parâmetros
- Usar test apenas para evaliação final

☐ Concluído?

2 Pré-processamento

- Pode usar bibliotecas externas
- Leitura dos textos
- Tokenização, normalização, stopwords
- Stemming/lematização (opcional)
- Documentar as escolhas no relatório

☐ Concluído?

3 Índice invertido (obrigatório do zero)

- Implementar o índice invertido do zero
- Escolher estruturas para dicionário e postings
- Armazenar TF, DF e tamanho dos documentos
- Medir: vocabulário, postings, tempo de indexação

☐ Concluído?

4 Busca lexical

- Escolher 1 baseline: Modelo Vetorial ou BM25
- Usar o índice invertido implementado
- Reaproveitar código do T1, se desejar
- Medir tempo médio das consultas

☐ Concluído?

2 Recuperação Inicial

Explorar técnicas de tolerância a erros e recuperação semântica.

5 Tolerância a erros

- Implementar: k-gramas → Jaccard
- → distância de edição
- Criar 10 consultas com erro ortográfico
- Comparar: original → com erro → corrigida
- Medir quanto da qualidade foi recuperada

☐ Concluído?

6 Recuperação semântica

- Usar embeddings densos com biencoder
- Ex:: Sentence-BERT / modelos equivalentes
- Pré-computar embeddings dos documentos
- Informar modelo, dimensão, similaridade e tempos

☐ Concluído?

7 Avaliação da recuperação

- Calcular: Precision@10, Recall@10, MAP@10, NDCG@10
- Manter métricas por consulta
- Guardar rankings e resultados em arquivos CSV

☐ Concluído?

3 Comparação e Hibridização

Comparar os sistemas e combinar seus resultados.

8 Lexical × Semântico

- Comparar os 2 sistemas nas mesmas consultas
- Identificar 3 consultas em que o semântico venceu
- Identificar 3 consultas em que o lexical venceu
- Mostrar Top-5 e explicar com evidências dos dados

☐ Concluído?

9 Busca híbrida

- Combinar rankings com RRF
- Ajustar k no conjunto dev: {10, 60, 100}
- Comparar: Lexical, Semântico e Híbrido
- Recalcular P@10, R@10, MAP@10 e NDCG@10

☐ Concluído?

4 Reranking

Refinar os resultados com um modelo mais avançado.

10 Reranking

- Pegar Top-50 do ranking híbrido
- Reordenar com cross-encoder
- Comparar: Lexical, Semântico, Híbrido e Híbrido + reranking
- Medir também o custo/tempo das etapas

☐ Concluído?

11 Análise do reranking

- 2 casos em que melhorou bastante
- 1 caso em que quase não mudou
- 1 caso em que piorou
- Mostrar quais documentos mudaram de posição

☐ Concluído?

5 RAG

Usar os documentos recuperados para gerar respostas com LLM.

12 Montagem do RAG

- Pipeline: Híbrido → reranking → seleção do contexto → LLM
- Usar os documentos recuperados como contexto
- Pode usar LLM aberta/local ou comercial
- Informar modelo, versão, parâmetros e custo (se houver)

☐ Concluído?

13 Experimento: sem RAG × com RAG

- Selecionar 10 consultas fixas
- Condição A: LLM recebe só a pergunta
- Condição B: LLM recebe pergunta + Top-5 documentos
- Mesmo modelo e mesmos parâmetros nas duas condições

☐ Concluído?

14 Avaliação do RAG

- Medir Recall@5 dos documentos enviados à LLM
- Avaliar respostas com notas 0–2
- Critérios: Fidelidade, Completude, Utilidade, Rastreabilidade
- Comparar métricas: sem RAG × com RAG

☐ Concluído?

15 Análise de falhas ponta a ponta

- 1 caso em que RAG ajudou muito
- 1 caso em que mudou pouco
- 1 caso em que não ajudou ou piorou
- Localizar a falha: recuperação, fusão, reranking, contexto ou geração

☐ Concluído?

6 Entrega Final

Organizar todos os arquivos e o relatório final.

16 Arquivos obrigatórios

- Código-fonte completo
- README com instruções, modelos e hardware
- CSVs: rankings, métricas, typo_experiment, rag_experiment
- Relatório em PDF (até 20 páginas)
- Seção "Uso de ferramentas de IA"

☐ Concluído?

Resumo dos Entregáveis

- ☐ Código
- ☐ README
- ☐ results_lexical.csv
- ☐ results_semantic.csv
- ☐ results_hybrid.csv
- ☐ results_reranking.csv
- ☐ metrics_per_query.csv
- ☐ typo_experiment.csv
- ☐ rag_experiment.csv
- ☐ Relatório PDF

Objetivo do roadmap: ajudar o grupo a visualizar o pipeline completo, acompanhar o progresso e evitar esquecer etapas obrigatórias.